



Chinese Society of Aeronautics and Astronautics
& Beihang University

Chinese Journal of Aeronautics

cja@buaa.edu.cn
www.sciencedirect.com



FULL LENGTH ARTICLE

Exploring crash induction strategies in within-visual-range air combat based on distributional reinforcement learning



Zetian HU^a, Xuefeng LIANG^a, Jun ZHANG^a, Xiaochuan YOU^{a,*},
Chengcheng MA^{b,c,d}

^a School of Aerospace Engineering, Tsinghua University, Beijing 100084, China

^b Xi'an Xiangteng Microelectronics Technology Co., Xi'an 710068, China

^c School of Software, Northwestern Polytechnical University, Xi'an 710129, China

^d Laboratory of Integrated Circuits and Microsystem Design for Aviation Technology, Xi'an 710068, China

Received 5 August 2024; revised 7 November 2024; accepted 11 February 2025

Available online 7 July 2025

KEYWORDS

Unmanned combat aerial vehicle;
Decision-making;
Distributional reinforcement learning;
Within-visual-range air combat;
Crash induction strategy

Abstract Within-Visual-Range (WVR) air combat is a highly dynamic and uncertain domain where effective strategies require intelligent and adaptive decision-making. Traditional approaches, including rule-based methods and conventional Reinforcement Learning (RL) algorithms, often focus on maximizing engagement outcomes through direct combat superiority. However, these methods overlook alternative tactics, such as inducing adversaries to crash, which can achieve decisive victories with lower risk and cost. This study proposes AlphaCrash, a novel distributional-reinforcement-learning-based agent specifically designed to defeat opponents by leveraging crash induction strategies. The approach integrates an improved QR-DQN framework to address uncertainties and adversarial tactics, incorporating advanced pilot experience into its reward functions. Extensive simulations reveal AlphaCrash's robust performance, achieving a 91.2% win rate across diverse scenarios by effectively guiding opponents into critical errors. Visualization and altitude analyses illustrate the agent's three-stage crash induction strategies that exploit adversaries' vulnerabilities. These findings underscore AlphaCrash's potential to enhance autonomous decision-making and strategic innovation in real-world air combat applications.

© 2025 Published by Elsevier Ltd on behalf of Chinese Society of Aeronautics and Astronautics. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

* Corresponding author.

E-mail address: youchaichuan@mail.tsinghua.edu.cn (X. YOU),

☆ Peer review under responsibility of Editorial Committee of CJA.



Production and hosting by Elsevier

1. Introduction

Due to advances in manufacturing and artificial intelligence technologies, Unmanned Aerial Vehicles (UAVs) have become prevalent in modern air warfare due to their superior performance.^{1,2} Compared to manned aerial vehicles, Unmanned Combat Aerial Vehicles (UCAVs) offer significant advantages:

(A) absence of risk to human life, (B) faster reaction times, (C) sensor equipment superior to the human eye, (D) greater stability, and (E) high overload capacity without concerns for human limitations.³ Consequently, UCAVs have become integral to contemporary military systems. Current air combat scenarios are categorized into Beyond-Visual-Range (BVR) and Within-Visual-Range (WVR) operations based on engagement distance.⁴ BVR air combat primarily utilizes radar detection and electronic warfare combined with airborne missiles, necessitating an advanced integrated combat system. In contrast, WVR air combat, or "dogfighting", involves close-range weapons such as guns and missiles, relying heavily on pilot skill. Effective WVR air combat demands rapid and agile maneuvers in close-range, complex, and dynamic engagements. The high-risk nature of such combat poses significant challenges, making intelligent decision-making algorithms crucial for maintaining high survivability rates and accomplishing various tasks.^{5,6}

Traditional air combat algorithms can be categorized into three branches: game theory-based algorithms, heuristic optimization algorithms, and expert knowledge-based algorithms. Game theory algorithms offer a systematic approach to analyzing decision-making problems involving multiple players with distinct goals. For instance, Park et al.⁷ applied differential game methods to autonomous maneuvering decision-making in WVR air combat. They used relative information, such as the relationships and velocities of both parties, to construct a score matrix that describes the advantages of each side. This matrix, combined with differential games, formed a hierarchical decision-making architecture. Similarly, Poropudas et al.⁸ introduced a multi-stage influence diagram game model to simulate pilots' decision-making processes in one-on-one air combat scenarios, integrating aircraft dynamics, pilot preferences, and uncertain conditions into a coherent framework. Heuristic optimization algorithms are designed to find good, feasible solutions for complex optimization problems within limited time frames.^{9,10} These algorithms transform air combat issues into optimization problems to find optimal solutions. McGrew et al.¹¹ proposed a method based on approximate dynamic programming, emphasizing that its success depends on feature development, reward shaping, and trajectory sampling. Expert knowledge-based methods use extensive data to inform automatic reasoning systems.^{12,13} Ernest et al.¹⁴ developed multiple parallel fuzzy inference engines to determine input-output connections based on mapping relationships, making real-time decisions for high-dimensional, complex problems that require continuous decision-making. Geng et al.¹⁵ combined expert systems with fuzzy Bayesian networks to create a hybrid strategy decision-making system, addressing the limitations of pure rule-based methods. Huang et al.¹⁶ established an air combat expert system by acquiring knowledge from extensive flight parameter data, improving exploration and exploitation efficiency by reconstructing the experience replay buffer to increase sample diversity.

As air warfare scenarios grow increasingly complex and the information capacity expands geometrically, there is an urgent need for more adaptable intelligent strategy formulation techniques. Traditional air combat algorithms often rely on predefined rules or precise models, which can yield good results to some extent. However, these methods face significant challenges in the highly unpredictable and dynamic air combat environment. Reinforcement Learning (RL), a branch of Arti-

ficial Intelligence, has achieved remarkable success in various fields, such as robotics, gaming, and autonomous driving.^{17,18} RL involves training agents to make decisions by rewarding desired actions, enabling them to learn optimal behavior through trial and error. Applying RL to air combat holds promise because it can help develop adaptive strategies that improve over time and under different conditions. In the realm of air combat, Deep Reinforcement Learning (DRL) has emerged as a transformative approach to UCAV maneuver decision-making. Air combat presents highly complex and non-linear challenges, further complicated by unpredictable maneuvers from adversaries. However, RL exhibits a high degree of adaptability to the environment and the ability to learn optimal decisions autonomously, making it highly suitable for solving air combat problems.

A pioneering study by Ma et al.¹⁹ utilized deep Q-learning for training agents for air combat maneuvers, demonstrating the potential of DRL in strategic decision-making for complex 3D environments. Xu et al.²⁰ further advanced this field by integrating Deep Belief Networks (DBN) with the Q-Learning algorithm in a UAV task decision model. DBN is used for unsupervised pre-training to extract deep feature information, while the Q-Learning algorithm is applied for the assessment and learning of decision-making behaviors. Yang et al.²¹ designed a comprehensive advantage assessment function that reflects situational awareness and distance, proposing a "basic-confrontation" training method that effectively improves training efficiency. Zhang et al.²² presented a maneuver decision-making method based on Final Reward Estimation and Proximal Policy Optimization (FRE-PPO), significantly enhancing the efficiency and performance of training in a self-play framework. Pope et al.²³ developed a hierarchical RL agent, PHANG-MAN, for air combat at DARPA's AlphaDogfight Trials, using a novel hierarchical DRL approach consisting of a high-level policy selector and a set of separately trained low-level policies specialized for excelling in specific regions of the state space. Chai et al.²⁴ proposed a hierarchical DRL framework for 6-DOF UCAV air-to-air combat, which separates decision-making into macro and micro levels, enhancing both tracking performance and combat effectiveness. Recently, Bae et al.²⁵ employed the Soft Actor-Critic (SAC) algorithm and the Long-Short-Term Memory (LSTM) network architecture to address inaccuracies in aircraft data, highlighting the importance of incorporating realistic limitations and sensor errors in model development.

Despite the extensive research conducted on DRL in air combat in recent years, several challenges remain, such as the considerable gap between research and real-world tasks and the oversimplification of simulation scenarios. These challenges can be summarized as follows. First, while many studies emphasize the complexity of the algorithms, they often overlook the mismatch between research results and practical applications. For instance, in real-world operations, enemy aircraft are frequently lured into specific areas and shot down, resulting in a global victory with a low-risk, low-cost strategy. Second, the focus on self-improvement through training often neglects the integration of advanced pilot experience. Additionally, there is a notable lack of expertise in maneuver design and state space characterization when applying RL to air combat. Finally, the vast differences between simulation platforms and real-world scenarios pose significant challenges for strategy transfer.

To address the challenges above, we propose a distributional reinforcement learning-based crash-inducing agent AlphaCrash for WVR air combat. The main contributions of this work can be summarized as follows:

- (1) A training and testing framework incorporating distributional RL is established, and improvements to the QR-DQN method are introduced into the air combat domain. This significantly enhances the intelligent agent's ability to counter uncertainties and diverse tactics.
- (2) Effective reward functions based on advanced pilots' experiences and real-world application scenarios are proposed, fusing distance, angle, velocity, boundary, and terminal evaluations to accelerate the learning process of an intelligent agent accurately.
- (3) Neural network-based characterization methods for flight information and discrete characterization methods for flight maneuvers were developed to ensure practical applicability and effectively address the demands of induced crash scenarios.
- (4) In extensive test scenarios simulating real air combat battles, the AlphaCrash method achieves an average win rate of 91.2% over 1 000 games, outperforming current high-level methods.

The organization of this paper is as follows: Section 2 outlines the problem formulation, detailing the aircraft dynamics and the WVR air combat scenario. Section 3 dives into the DRL agent, discussing the principles of RL, the application of distributional RL, and the training framework. Section 4 presents the experimental settings, results, and analysis, demonstrating the effectiveness of our approach in various combat scenarios. Finally, Section 5 concludes the paper, summarizing the findings and discussing possible future directions.

2. Problem formulation

In this section, we will provide definitions of aircraft dynamics, WVR air combat scenarios, and rules, and subsequently formalize the WVR air combat problem.

2.1. Aircraft dynamics

To effectively model air combat dynamics, the motion of an aircraft in a one-on-one engagement scenario must be accurately represented. Typically, aircraft motion is modeled with six degrees of freedom for rigid body dynamics. However, for higher-level autonomy tasks, simplified aircraft models are practical to reduce complexity. When faster aircraft dynamics (e.g., angular rates with respect to the body axis) are controlled by well-designed inner-loop controllers, treating these dynamics as first-order lags is sufficient. This study utilizes 3D Dubin's aircraft models²⁶ with the following equations:

$$\begin{cases} \dot{x} = v \cos \eta \sin \psi \\ \dot{y} = v \cos \eta \cos \psi \\ \dot{z} = v \sin \eta \\ \dot{\eta} = \frac{g}{v} (n_z \cos \mu - \cos \eta) \\ \dot{\psi} = \frac{g n_x \sin \mu}{v \cos \eta} \end{cases} \quad (1)$$

Here, x, y , and z denote the position of the aircraft, with $\dot{x}, \dot{y}$, and $\dot{z}$ representing their rates of change. The variable v represents the aircraft's speed. The angles η, ψ , and μ are the track angle, heading angle, and roll angle, respectively. $\dot{\eta}$ and $\dot{\psi}$ denote the rates of change of the track and heading angles. The term n_z represents the normal overload.

2.2. WVR air combat scenario

WVR air combat, also known as dogfighting, involves maneuvering to position one's aircraft to chase the tail of the opponent while avoiding being targeted in the same way.

We consider a one-on-one air combat scenario involving red and blue aircraft, both exhibiting similar dynamic behaviors and striving for air dominance. The blue aircraft follows a predefined strategy, whereas the red aircraft's strategy is adaptable and improves through training. As illustrated in Fig. 1, the UAV's strategy in such scenarios has two primary objectives: attack and defense. Offensively, the UAV should pursue the enemy by maintaining a high Antenna Train Angle (ATA). Defensively, it should maximize the Aspect Angle (AA) to minimize its profile to the enemy.

2.3. Combat rule

Weapons used in WVR air combat typically have a restricted range, limiting engagement to within their designated reach. However, within this range, aircraft can cause considerable damage.²³ To simulate this situation, we have devised specific rules for decreasing aircraft health. Specifically, an agent inflicts damage and reduces its opponent's health whenever the opponent is within the agent's Weapons Engagement Zone (WEZ).

The WEZ encompasses all points within a 3 000 m radius that also fall inside a spherical cone with a 3° aperture extending from the aircraft's nose, as shown in Fig. 2. Each aircraft begins with 100 health points. For every second an opponent stays within the WEZ, it loses 20 health points. Importantly, an aircraft will crash if its altitude falls below 100 m, a critical point given that this thesis focuses on inducing the enemy to crash. Additionally, a draw occurs if the distance between the two aircraft is less than 20 m, resulting in a collision.

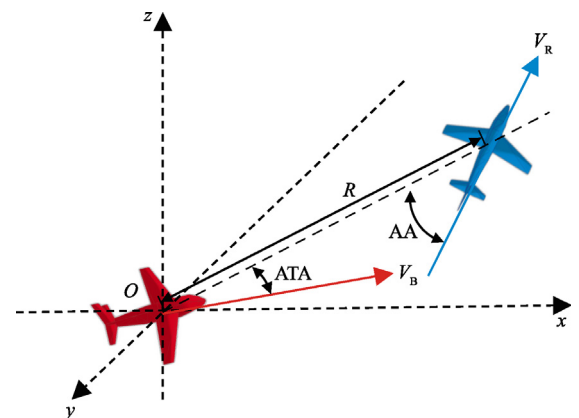


Fig. 1 One-to-one WVR air combat situation.

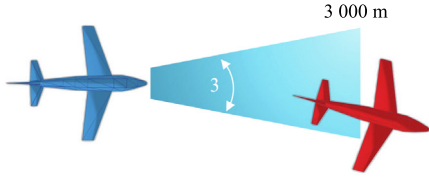


Fig. 2 Weapon engagement zone.

2.4. Problem statement

The agent can be defined as an optimization function F , where the input is a sequence of n -dimensional aircraft environment-aware states s'_n over the past τ time steps, and the output is an m -dimensional action a'_m of the aircraft control, as shown in Eq. (2). At each time step, the agent's action output a'_m is sent to the simulation platform to modify the attitude and flight path of the aircraft. The simulation platform then updates the state values of both aircraft in the current environment based on these actions in conjunction with the aircraft dynamics model.

$$a'_m = F(s'_n{}^{t-\tau+1:t}) \quad (2)$$

3. Distributional RL-based agent for air combat

3.1. DRL agent

In essence, an RL agent learns to maximize a numerical reward signal by selecting actions based on the current state of its environment. The environment provides reward signals, and learning occurs iteratively through trial and error. Unlike supervised learning, which uses labeled examples from an external supervisor, and unsupervised learning, which seeks hidden data structures, RL involves managing the tradeoff between exploration and exploitation.²⁷ The RL agent must explore new actions to find potentially higher rewards while exploiting known actions to maximize immediate rewards.²⁸

The decision-making problem in WVR air combat can be represented as a Markov decision process $M = (S, A, T, p(s_0), r, \gamma)$, specifically including a set of states S , comprising various air combat information related to both the self and the opponent; a set of actions A , including various decisions about control maneuvers; a transition function $T(s_{t+1}|s_t, a_t)$, mapping state-action pairs at time t to state distributions at time $t + 1$; an initial state distribution $p(s_0)$, probability distributions for Red and Blue at initialization (e.g., position and attitude); a reward function $r : S \times A \rightarrow R$, calculating immediate rewards based on chosen actions and achieved states; and a discount factor $\gamma \in (0, 1]$, used to balance the significance of future rewards.

The objective of the WVR agent is to identify a policy $\pi(a_t|s_t)$, which maps states to action distributions. The agent's trajectory over L iterations is denoted as $\tau = (s_0, a_0, s_1, a_1, \dots, s_L, a_L)$. The probability $p_\pi(\tau)$ that τ occurs, given the strategy π :

$$p_\pi(\tau) = p(s_0) \prod_{t=0}^{L-1} \pi(a_t|s_t) T(s_{t+1}|s_t, a_t) \quad (3)$$

Then the expected value of the RL objective $J(\pi)$ can be written as:

$$J(\pi) = \mathbb{E}_{\tau \sim p_\pi(\tau)} \left[\sum_{t=0}^L \gamma^t r(s_t, a_t) \right] \quad (4)$$

A straightforward method to optimize the RL objective is to estimate the value function $V^\pi(s_t)$. This function represents the expected cumulative reward of the policy π when starting from the state s_t . Alternatively, the state-action value function, denoted as $Q^\pi(s_t, a_t)$, can be used, which includes both the state s_t and the action a_t :

$$V^\pi(s_t) = \mathbb{E}_{\tau \sim p_\pi(\tau|s_t)} [Q^\pi(s_t, a_t)] \quad (5)$$

$$Q^\pi(s_t, a_t) = r(s_t, a_t) + \mathbb{E}_{s_{t+1} \sim T_\pi(s_t, a_t)} [V^\pi(s_{t+1})] \quad (6)$$

When the chosen strategy is $\pi(a_t|s_t) = \delta(a_t = \arg \max_{a_t} Q(s_t, a_t))$, it corresponds to a Q-learning method. Substituting this strategy into Eqs. (5) and (6) yields the optimal Q-function for the method:

$$Q^*(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim T(s_t, a_t)} \left[\max_{a_{t+1}} Q^*(s_{t+1}, a_{t+1}) \right] \quad (7)$$

Parameterizing the Q-function using a deep convolutional network and training it is the classical DQN method, which can achieve excellent performance on Atari games.²⁸

3.2. Distributional RL

Previous value-based RL techniques aim to represent cumulative returns through expected values. This is exemplified in the classic DQN algorithm, where the network output predicts the expected return for each action, leading to a considerable loss of full distribution details. To obtain more detailed information, the value distributional RL has been developed. It focuses on capturing the full distribution of the random variable of cumulative returns rather than merely its expected value. The Quantile Regression Deep Q-Network (QR-DQN) is a value-distribution RL approach that is particularly suited to the highly uncertain air combat domain.²⁹

QR-DQN improves upon the classic DQN approach by integrating quantile regression to estimate the full distribution of state-action values, not just their expected values, as shown in Fig. 3. Using a multi-head output network, QR-DQN estimates multiple quantiles, thereby offering a more comprehensive depiction of Q-value distributions.

Algorithm 1 presents the pseudocode for the entire QR-DQN procedure. QR-DQN employs a quantile regression loss function to train the network. The target value y_j is given by: $y_j = r_j + \gamma \sum_{i=1}^N Q^-(\phi_{j+1}, a^*; \theta^-)[\tau_i]$, where r_j represents the reward, γ is the discount factor, ϕ_{j+1} represents the next state, a^* is the optimal action selected by the current network, and θ^- corresponds to the parameters of the target network. Defining the quantile regression error as $\delta_{ij} = y_j - Q(\phi_j, a_j; \theta)[\tau_i]$, the quantile regression loss function is:

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^N \rho_\kappa(\delta_{ij}) \quad (8)$$

where ρ_κ is the quantile Huber loss function, defined as:

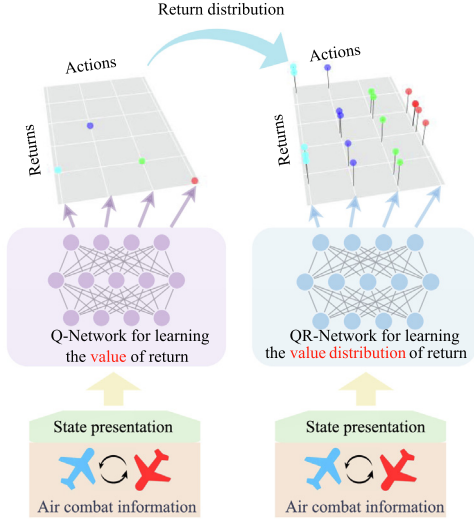


Fig. 3 A schematic illustration of the improvements in QR-DQN, which estimates the full distribution of state-action values using quantile regression.

$$\begin{cases} \rho_\kappa(u) = |\kappa - \mathbb{I}(u < 0)| \cdot L_\kappa(u) \\ L_\kappa(u) = \begin{cases} \frac{1}{2}u^2, & \text{if } |u| \leq \kappa \\ \kappa(|u| - \frac{1}{2}\kappa), & \text{otherwise} \end{cases} \end{cases} \quad (9)$$

where κ is the threshold parameter for the Huber loss, and $\mathbb{I}(\cdot)$ is the indicator function. The algorithm updates the network parameters θ by minimizing this loss function. In every C step, the algorithm copies the parameters of the current network to the target network, i.e., $\theta^- \leftarrow \theta$.

Algorithm 1. QR-DQN

```

1: Initialize replay buffer  $D = \{\}$ 
2: Initialize quantile action-value network  $Q$  with random weights  $\theta$ 
3: Initialize target network  $Q^-$  with weights  $\theta^- \leftarrow \theta$ 
4: Set number of episodes  $M$ , number of iterations  $T$ , target update frequency  $C$ 
5: Set number of quantiles  $N$  and quantile levels  $\tau_i = \frac{2i-1}{2N}$  for  $i = 1, 2, \dots, N$ 
6: for episode = 1, 2, ...,  $M$  do
7:   Initialize sequence  $s_1 = \{x_1\}$  and preprocess  $\phi_1 = \phi(s_1)$ 
8:   for  $t = 1, 2, \dots, T$  do
9:      $a_t = \begin{cases} \text{random action,} & \text{with probability } \epsilon \\ \arg \max_a \sum_{i=1}^N Q(\phi_t, a; \theta)[\tau_i], & \text{otherwise} \end{cases}$ 
10:    Execute  $a_t$ , observe reward  $r_t$  and next state  $s_{t+1}$ , preprocess  $\phi_{t+1} = \phi(s_{t+1})$ 
11:    Store transition  $(\phi_t, a_t, r_t, \phi_{t+1})$  in  $D$ 
12:    Sample minibatch of transitions  $(\phi_j, a_j, r_j, \phi_{j+1})$  from  $D$ 
13:     $a^* = \arg \max_a \sum_{i=1}^N Q(\phi_{j+1}, a; \theta)[\tau_i]$ 
14:     $y_j = r_j + \gamma \sum_{i=1}^N Q^-(\phi_{j+1}, a^*; \theta^-)[\tau_i]$ 
15:    Update  $\theta$  by minimizing the loss:

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^N \rho_{\tau_i}(y_j - Q(\phi_j, a_j; \theta)[\tau_i])$$

    where  $\rho_\kappa(u) = |\kappa - \mathbb{I}(u < 0)| \cdot L_\kappa(u)$ 
16:   Every  $C$  steps, update target network:  $\theta^- \leftarrow \theta$ 
17:   end for
18: end for

```

Experience replay stores action-generated data tuples in a buffer to enhance training efficiency. While standard experience replay randomly samples from the buffer, certain samples contribute more significantly to model convergence. Prioritized Experience Replay (PER) optimizes sampling to accelerate model convergence. Eq. (10) defines δ as the temporal difference error between predicted and target Q-values.

$$\delta = r + \max_{a_{t+1}} Q(Q_{t+1}, A_{t+1}) - Q(Q_t, A_{t+1}) \quad (10)$$

Samples with larger δ values, indicating greater prediction errors, receive higher training priority. Eq. (11) defines the sampling probability, with α controlling the error magnitude's influence.

$$\begin{cases} P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha} \\ p_i = |\delta_i| + \epsilon \end{cases} \quad (11)$$

PER can bias model convergence during gradient descent optimization. Importance sampling weights w_i are introduced to correct this bias. Eq. (12) defines w_i , where β controls the bias correction strength, with $\beta = 1$ providing full compensation.

$$w_i = \left(\frac{1}{N} \times \frac{1}{P(i)} \right)^\beta \quad (12)$$

3.3. Distributional RL in air combat

Distributional RL offers unique advantages for decision-making in dynamic and adversarial environments like air combat. Unlike traditional RL methods that rely on expected return values, distributional RL models the entire distribution of potential outcomes. This provides the agent with a granular understanding of both likely and extreme scenarios, allowing for better handling of uncertainties inherent in air combat, such as adversary unpredictability and environmental variability.

The use of Quantile Regression in the QR-DQN framework further strengthens the agent's decision-making capabilities by enabling a detailed analysis of outcome probabilities. This ensures the agent can not only identify the most optimal actions under average conditions but also adapt strategies to minimize risks in worst-case scenarios. For instance, in a dog-fight, the ability to predict the likelihood of crashes or boundary violations provides a strategic edge in inducing adversarial errors. By modeling potential outcomes across a spectrum of quantiles, the algorithm avoids over-reliance on mean-value approximations, which can obscure critical tactical insights in high-stakes environments. For example, in low-altitude dog-fights, understanding the tail-end probabilities of altitude violations allows the agent to craft maneuvers that maximize adversarial errors while minimizing its own risk.

Moreover, distributional RL facilitates the discovery of innovative and non-intuitive tactics, such as crash induction. By analyzing the full range of outcomes, the agent can identify subtle adversary weaknesses that might remain undetected in traditional RL or heuristic optimization methods. This adaptability makes distributional RL particularly suitable for developing robust, real-time decision-making systems in highly dynamic and uncertain environments.

3.4. RL framework for one-on-one air combat

In the realm of WVR air combat maneuver decision-making, a robust framework is essential for training RL agents. Our framework, as shown in Fig. 4, integrates the states of both the UAV and its enemy to generate a comprehensive state description of the combat environment, which is then fed to the agent. The RL model evaluates the UAV's situational advantage and uses this assessment as a reward to guide the agent.

The framework operates through several interconnected processes. First, the environment continuously collects comprehensive state information from both red and blue UAVs, including their positions, velocities, orientations, and health status, which are processed to form a complete representation of the current combat situation. The RL agent then processes this state information through its trained policy network, leveraging distributional RL to consider the full range of potential outcomes when selecting actions. These selected actions are executed in the combat environment, updating the red UAV's state based on both the actions and aircraft dynamics, while the enemy blue UAV simultaneously adjusts its state according to its predefined strategy.

During these interactions, the environment calculates multiple reward components based on distance, angle, speed, and boundary conditions, which are weighted and combined according to the current engagement phase to provide a comprehensive learning signal. This continuous interaction allows the RL agent to gather various states, actions, and rewards, storing them as experience tuples and periodically sampling them to update its policy through the QR-DQN algorithm. Consequently, the UAV's action outputs progressively optimize, enabling self-learning of increasingly effective air combat maneuvers.

3.5. State representation

When creating state representations for RL in air combat scenarios, the variables and their computation can be organized and explained using standard mathematical symbols typical in aviation and combat modeling. The state variables are generally categorized into two types: those containing information specific to the aircraft and those comprising relative information between the two aircraft. Table 1 provides a detailed analysis.

By employing the variables above, the air combat scenario involving two aircraft can be comprehensively described at any

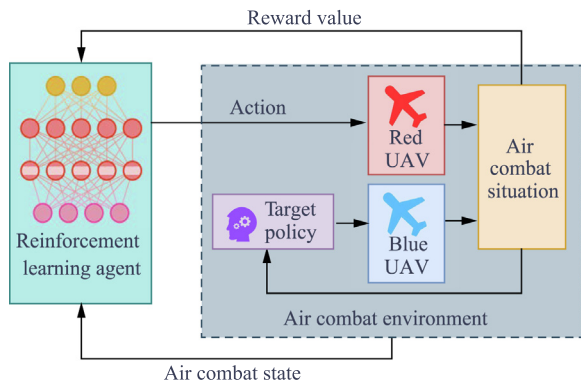


Fig. 4 Air combat agent training and testing framework.

given moment. This description captures the physical properties and dynamic behaviors of the aircraft, including their relative positions and orientations, which are crucial for devising effective combat strategies. Subsequently, normalization is conducted on the state variables. This essential step scales all feature values to a consistent range of -1 to 1 , thereby accelerating the learning process and improving the model's convergence rate.

3.6. Action representation

Selecting the action space is crucial when tackling air combat simulation scenarios with RL. The aircraft's action space comprises three distinct actions: Acceleration (Acc), Yaw rate (Yaw), and Pitch rate (Pitch). We represent each action as a discrete variable and select the values of all three actions simultaneously at each time step. The comprehensive explanation is as follows.

- (1) Acceleration: This function manages the aircraft's acceleration or deceleration along its Z axis, enabling rapid approach or retreat maneuvers. The acceleration action is designed with nine distinct values, ranging from -30 to 90 m/s^2 , specified as the array $\text{ACC}[9] = \{-30, -20, -10, 0, 10, 20, 40, 60, 90\}$.

Table 1. State representation for RL air combat scenario.

Category	Variable	Symbol	Description
Position and kinematic state variables for each aircraft i	Position vector	ξ_i	Position of aircraft i
	Direction angles	χ_i	Aircraft i 's direction angles including yaw, roll, and pitch
	World direction vector	w_i	Aircraft i heading in 3D space
	Speed	v_i	Speed of aircraft i
	Acceleration	a_i	Rate of change of speed for aircraft i
Relative position and directional state variables	Yaw rate	$\omega_{y,i}$	Rate of change of yaw angle for aircraft i
	Pitch rate	$\omega_{p,i}$	Rate of change of pitch angle for aircraft i
	Health status	H_i	Health status of aircraft i
	Relative position vector	$\Delta\xi$	$\xi_2 - \xi_1$
	Relative distance	d	Relative distance between aircraft, calculated as $d = \Delta\xi $
	Relative angles	σ	Angles obtained by projecting $\Delta\xi$ onto the XY , YZ , and XZ planes
	Angular difference	ζ_i	The angular difference between the aircraft i direction and the relative position, calculated as $\zeta_i = \cos^{-1} \left(\frac{w_i \cdot \Delta\xi}{ w_i \Delta\xi } \right)$

- (2) **Yaw Rate:** This regulates the aircraft's rotation around its Y axis, which is vital for changing the UAV's path. The yaw action is designed into nine distinct values, from -30 to $30^\circ/\text{s}$, noted as $\text{Yaw}[9] = \{-30, -21, -12, -5, 0, 5, 12, 21, 30\}$.
- (3) **Pitch Rate:** This governs the aircraft's rotation around its X axis, which is crucial for tracking or evading threats. Similar to the yaw action, the pitch action is designed to have nine discrete values ranging from -60 to $60^\circ/\text{s}$, indicated as $\text{Pitch}[9] = \{-60, -34, -18, -8, 0, 8, 18, 34, 60\}$.

Dividing each action into nine dimensions is intended to strike a balance between efficient execution and implementation precision. This nine-dimensional discretization equips the algorithm with essential operational choices, enabling it to investigate various strategies while maintaining computational demands at a manageable level.

3.7. Reward function

In applying RL to WVR scenarios, the reward function plays a crucial role in shaping the learning process and behavior of agents by evaluating the desirability of their actions within a simulated environment. In air combat simulations, reward functions are designed to promote realistic and strategically sound behaviors in fighter jets. These functions balance offense and defense, adhere to safe operational boundaries, and optimize combat outcomes. For WVR air combat, specific reward functions have been developed to induce enemy crashes while protecting one's aircraft effectively. These functions comprise terminal and non-terminal rewards. Terminal rewards primarily evaluate combat outcomes, whereas non-terminal rewards guide the aircraft toward advantageous tactics during combat. These aspects are elaborated further.

3.7.1. Terminal reward

Terminal rewards are used to measure the final outcome of a combat, specified as follows.

$$R_T = \begin{cases} -1, & \text{if } H_{\text{self}} < H_{\text{enemy}} \text{ or } C_{\text{self}} < C_{\text{enemy}} \\ 1, & \text{if } H_{\text{enemy}} < H_{\text{self}} \text{ or } C_{\text{enemy}} < C_{\text{self}} \end{cases} \quad (13)$$

where C_{self} and C_{enemy} represent the crash states of ourselves and the enemy, respectively (1 if crashed, otherwise 0), H_{self} and H_{enemy} represent the health points of ourselves and the enemy, respectively.

3.7.2. Non-terminal reward

Non-terminal rewards include four types: distance reward, angle reward, speed reward, and boundary reward.

Distance Reward: This reward quantifies the distance between the aircraft and the enemy, providing incentives based on how close they are, as defined below.

$$R_d = \begin{cases} 1 - \frac{d}{d_{\text{ideal}}}, & \text{if } d > d_{\text{ideal}} \\ e^{-\left(\frac{d}{d_{\text{ideal}}} - 1\right)} - 1, & \text{if } d \leq d_{\text{ideal}} \end{cases} \quad (14)$$

where d signifies the current relative distance between aircraft, and d_{ideal} represents the ideal distance. This design aims to leverage the enemy units' continuous pursuit behavior towards

our units, encouraging the enemy to follow us closely. The strategy is to steer the enemy into low-altitude zones, leading to a loss of control and crash.

Angle Reward: Angle reward evaluates the tactical advantage between the aircraft and the enemy, awarded based on the size of the angle, specified as follows.

$$R_a = 1 - \frac{|\theta_{\text{ATA}}| + |\theta_{\text{AA}}|}{360} \quad (15)$$

where θ_{ATA} represents the antenna train angle, and θ_{AA} represents the aspect angle. R_a provides incentives to the agent for achieving the optimal dogfighting geometry where both the track angle and adverse angle are minimized. It also imposes penalties on the agent if it permits its opponent to achieve the same. While our primary objective is to cause the enemy to crash, maintaining optimal dogfighting geometry helps minimize damage.

Speed Reward: Speed reward compares the speed difference between the aircraft and the enemy, awarded based on the speed advantage, specified as follows.

$$R_v = \begin{cases} \frac{v_{\text{self}} - v_{\text{enemy}}}{700}, & \text{if } v_{\text{self}} \leq v_{\text{enemy}} \\ \frac{v_{\text{self}} - v_{\text{enemy}}}{300}, & \text{if } v_{\text{self}} > v_{\text{enemy}} \end{cases} \quad (16)$$

where v_{self} represents our speed, and v_{enemy} represents the enemy's speed. To induce the enemy to keep chasing us, it is essential for us to be faster than the enemy in order to avoid being overtaken and engaging in a dogfight, which could result in damage.

Boundary Reward: Primarily used to induce enemy crashes while preventing one's own. Boundary reward is divided into self-boundary reward and enemy-boundary reward, measuring the gap between the aircraft and the enemy with the warning height.

$$\begin{cases} R_b^{\text{self}} = \begin{cases} \frac{h_{\text{self}}}{h_{\text{warning}}} - 1, & \text{if } h_{\text{self}} < h_{\text{warning}} \\ 0, & \text{if } h_{\text{self}} > h_{\text{warning}} \end{cases} \\ R_b^{\text{enemy}} = \begin{cases} \frac{h_{\text{enemy}}}{h_{\text{warning}}} - 1, & \text{if } h_{\text{enemy}} < h_{\text{warning}} \\ 0, & \text{if } h_{\text{enemy}} > h_{\text{warning}} \end{cases} \end{cases} \quad (17)$$

where h_{self} and h_{enemy} represent the altitudes of ourselves and the enemy, respectively, and h_{warning} represents the warning height.

The reward functions R_b^{self} and R_b^{enemy} are constructed to control the altitude strategies of aircraft in combat. The self-boundary reward, R_b^{self} , imposes a penalty on our aircraft when its altitude h_{self} falls below a specified warning height h_{warning} . This penalty intensifies as the altitude descends further below this threshold, motivating the aircraft to climb to a safer level. On the other hand, the enemy-boundary reward, R_b^{enemy} , promotes actions that either maintain or lower the enemy's altitude beneath this warning height, thereby increasing the chance of an enemy crash. Both rewards are nullified when their respective aircraft's altitudes exceed the warning height, signifying no immediate danger or need for altitude correction. This combined reward framework effectively guarantees safety through altitude management while also exerting strategic pressure on the enemy, making it a vital part of the model's learning to balance defensive and offensive maneuvers.

The overall non-terminal reward is defined as a weighted sum of four component rewards. Crucially, the weights

assigned to these components dynamically adjust based on the current distance between the agent and the enemy aircraft, as illustrated in the following reward function:

$$R_{NT} = \begin{cases} 0.6R_d + 0.3R_a + 0.1R_v \\ +0.5(R_b^{\text{self}} + R_b^{\text{enemy}}), & \text{if } d \leq d_{\text{close}} \\ 0.7R_a + 0.2R_v \\ +0.5(R_b^{\text{self}} + R_b^{\text{enemy}}), & \text{if } d > d_{\text{close}} \end{cases} \quad (18)$$

where d represents the current distance, and d_{close} denotes the distance that two aircraft are within close range.

The adaptive weighting scheme demonstrates a sophisticated understanding of air combat dynamics, reflecting evolving priorities across various engagement stages. At long ranges where $d \leq d_{\text{close}}$, the primary objectives are to reduce the distance to the enemy, maintain speed superiority, and guide them to lower altitudes. Consequently, distance reward is assigned a high weight, while angle and speed rewards receive lower weights, indicating that precise positioning and speed control are less critical in the initial engagement phase. At close range ($d > d_{\text{close}}$), the strategy shifts dramatically. Distance-based rewards are eliminated, as the focus shifts from closing in on the enemy to maintaining tactical superiority. The increased weight of the angle reward and the speed reward become paramount, emphasizing the critical importance of positioning and energy management at close range. Boundary rewards maintain their high weight across all distances, underscoring the consistent priority of altitude management for self-preservation and inducing enemy crashes. This dynamic reward structure guides the RL agent through complex air combat phases, from the initial approach to close-range engagement, while prioritizing the overarching goal of inducing enemy crashes through strategic maneuvering and altitude management. The adaptability of the reward function enables the agent to seamlessly transition between tactical priorities as the engagement evolves, fostering a robust and versatile combat strategy.

3.8. High-level enemy strategy based on pilot experience

Given the extensive state space of the air combat model, training an uninformed UAV against itself directly leads to considerable ineffective exploration and thus low efficiency. Incorporating an enemy policy into RL-based air combat strategy establishes an evolving and challenging environment that significantly enhances learning, adaptability, and strategic depth.³⁰ This approach leverages competitive pressure and adaptive strategy to develop more advanced and effective maneuver techniques.

In training for confrontations, we employ a high-level enemy strategy based on pilot experience. Specifically, this experience is used to assess air combat advantages. The enemy's maneuver policy utilizes a robust decision-making framework based on statistical principles. This framework shows strong robustness, with simulations that demonstrate its superiority over the traditional min-max method.³¹ The framework of the strategy involves evaluating all actions within the maneuver library under the current conditions, solving the trajectory differential equations using the Runge-Kutta algorithm to predict potential next-phase scenarios, assessing

advantages between us and the enemy based on pilot experience for each scenario, and ultimately choosing the action that maximizes the advantage. The detailed process of the enemy policy depicted in Fig. 5 is as follows.

Based on the current information about the aircraft at time t , all control commands in the action library are entered into the Runge-Kutta differential computation model to search the action space. Calculate the advantages for both the enemy and ourselves in each scenario in the action and derive the advantage set:

$$T^{t+\Delta t} = \{T_1^{t+\Delta t}, T_2^{t+\Delta t}, \dots, T_n^{t+\Delta t}\}, \quad i = 1, 2, \dots, n \quad (19)$$

where n is the size of the action space.

The air combat advantages based on pilot experience consist of two parts: angular advantage and distance advantage, combined multiplicatively for the final evaluation. The angular advantage value, related to the UAV direction and relative position of both aircraft, is expressed as:

$$T_a = \frac{|\theta_{ATA}| + |\theta_{AA}|}{360} \quad (20)$$

A smaller T_a indicates a greater advantage. When $T_a = 0$, our aircraft is directly behind the enemy, providing the greatest advantage. Conversely, when $T_a = 1$, the enemy is directly behind us, providing the least advantage. Distance advantage must account for angular advantage, as it varies with different angles. The air combat experience formula is as follows, where a smaller T_d indicates a greater advantage:

$$T_d = \begin{cases} 1 - \frac{d_{\text{engage}} - d}{d_{\text{engage}}}, & \text{if } T_a < 0.5 \text{ and } d \leq d_{\text{engage}} \\ 1 - \frac{d_{\text{engage}} - d}{d}, & \text{if } T_a < 0.5 \text{ and } d > d_{\text{engage}} \\ 3 + \frac{d_{\text{engage}} - d}{d}, & \text{if } T_a \geq 0.5 \text{ and } d > d_{\text{engage}} \\ 3 + \frac{d_{\text{engage}} - d}{d_{\text{engage}}}, & \text{if } T_a \geq 0.5 \text{ and } d \leq d_{\text{engage}} \end{cases} \quad (21)$$

where d_{engage} is the engagement distance. When angular advantage exists ($T_a < 0.5$), shorter distances are better. Conversely, when at an angular disadvantage ($T_a \geq 0.5$), greater distances are better.

The overall situational advantage function T is calculated by multiplying T_a and T_d .

$$T = T_a \times T_d \quad (22)$$

A smaller T value indicates a more favorable situation for us. In the last step, select and execute the optimal action based on the advantages between us and the enemy in all scenarios.

4. Experiment

4.1. Experimental settings

4.1.1. Initial setup

To enhance the strategy's robustness and adapt to more varied environments, we established a wide initial range that encompasses both advantages and disadvantages. During training, the aircraft's initial positions are randomly assigned each time the environment is reset. This initial range includes relative distances between 500 and 10 000 m, altitude between 7 000 and 10 000 m, heading angles from 0° to 360° for both red and blue aircraft, and a speed of 300 m/s.

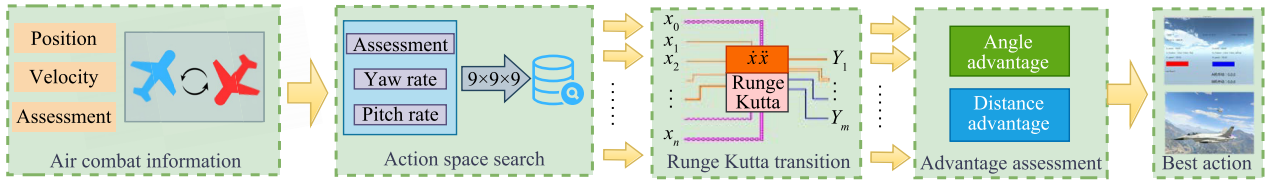


Fig. 5 High-level enemy policy based on pilot experience.

To test our methods, we developed 10 preliminary scenarios to assess the effectiveness and versatility of the strategies. These scenarios cover a wide range of initial conditions, ensuring a comprehensive evaluation. Each scenario differs in relative positions, heading angles, and altitudes, ensuring a detailed analysis of the strategies' capabilities across various combat scenarios. The scenarios used for testing are presented in Table 2. Scenarios 1–4 comprise pairs where advantages alternate between the Red and the Blue sides. Scenarios 5 and 6 represent equitable conditions, with both aircraft starting at the same altitude but at an increased distance (10 km) relative to other scenarios.

Fig. 6 visualizes the heading angles of both Red and Blue aircraft. These visualizations help us better understand the initial conditions and show the diversity of scenarios considered in our testing phase. For instance, Fig. 6(a) shows both aircraft at 0° heading, while Fig. 6(c) shows the Blue aircraft at 0° and the Red aircraft at 180° .

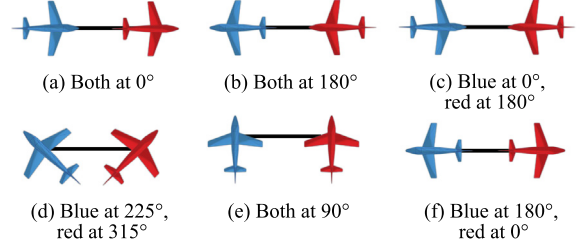


Fig. 6 Visualization of different heading angles for Red and Blue aircraft.

4.1.2. Comparison methods

To evaluate the effectiveness of the proposed AlphaCrash framework, we compare it against several state-of-the-art RL methods, all trained against the high-level enemy strategy based on pilot experience. These methods include the standard Deep Q-Network (DQN)³² and Proximal Policy Optimization (PPO)³³ algorithms, which have been widely used in air combat strategy development due to their strong performance in convergence and sample efficiency. In addition, we developed enhanced versions of these algorithms, denoted as DQN* and PPO*, to align these methods with the specific challenges of WVR air combat. As illustrated in Section 3, these enhancements included integrating a high-level enemy training framework, optimized reward functions emphasizing crash induction, expanded state representation with critical air combat variables, and refined action spaces for precise maneuvering.

4.1.3. Parameter settings

All experiments were conducted on a machine equipped with Intel Xeon Gold 6230 CPUs, and NVIDIA GeForce RTX

3090 GPUs. The implementation utilized Python 3.9 and PyTorch 1.10. The reward function parameters are defined as: $d_{\text{ideal}} = 10$ km, $h_{\text{warning}} = 5$ km, and $d_{\text{close}} = 3$ km, $d_{\text{engage}} = 3$ km. To ensure statistical robustness, each algorithm underwent training with 5 different random seeds. Table A1 of Appendix A presents the hyperparameter configurations for AlphaCrash and the baseline algorithms (DQN* and PPO*). The simulation parameters are listed in Table B1 of Appendix B.

4.2. Simulation results

In this section, we first demonstrate the effectiveness of our proposed methods through quantitative comparisons. Subsequently, we provide intuitive comparisons of the training process and results for each method through reward function visualization.

4.2.1. Quantitative results

The trained agents were tested in the 10 scenarios set up in Section 4.1.1, running 100 rounds per scenario for a total of 1 000 rounds. Table 3 presents the results of the performance evaluation in various scenarios. Our approach consistently overcame high-level enemy strategy based on pilot experience across every scenario, achieving an average win rate of

Table 2 Testing scenarios with varying initial conditions.

Scenario	Distance/km	Red heading($^\circ$)	Blue heading($^\circ$)	Red altitude(km)	Blue altitude(km)
1	1	0	0	7	7
	1	180	180	7	7
2	1	0	180	6	7
	1	0	180	7	6
3	1	225	315	7	6
	1	225	315	6	7
4	1	90	90	7	6
	1	90	90	6	7
5	10	180	0	7	7
6	10	0	180	7	7

91.2%. The rest of the matches ended in draws, with no losses observed. In Scenarios 1–4, which include both advantageous and disadvantageous conditions, AlphaCrash prevailed over the enemy with notable average win rates of 92.9% (minimum of 89% and maximum of 97%). In Scenarios 5 and 6, representing balanced conditions at a greater distance, our approach maintains a strong win rate of 74% and 95%, respectively, against the enemy. It should be noted that the enemy algorithms can outscore the project researchers in head-to-head play on the simulator platform. Compared to DQN* and PPO*, AlphaCrash outperformed the other two methods in all scenarios, demonstrating superior win rates and overall effectiveness. Across all scenarios, AlphaCrash demonstrated a win rate of 91.2%, with no losses and 88 draws. In comparison, DQN* achieved a win rate of 66.3% with 161 losses and 176 draws, while PPO* showed a significantly lower win rate of 5.9%, with 295 losses and 646 draws.

For a more in-depth performance analysis, we classified the outcomes into six categories: draw by collision, lose by crash, win by crash, draw by blood, lose by blood, and win by blood, as illustrated in Fig. 7. AlphaCrash demonstrated a consistent ‘win by crash’ result across all 6 scenarios, achieving win rates exceeding 90% in 5 scenarios. This result indicates the successful development of a highly reliable and efficient crash-inducing tactic. In the remaining matches, we were never defeated by the opponent but rather ended in a draw due to collisions, which further validated the stability of AlphaCrash. Our agent won’t crash itself out of control because of failing to induce the enemy. Conversely, the DQN agent’s test results exhibited a certain number of failed crash inductions, leading to its own crashes. Additionally, the DQN* agent’s win rate was significantly lower compared to AlphaCrash and included a number of ‘lose by blood’ where it could not induce a crash in the opponent. This indicates that although the DQN* agent has learned some crash induction strategies, they are unstable and not adequately effective. The PPO* agent’s test results showed very poor performance, achieving ‘win by blood’ or ‘win by crash’ only occasionally.

The quantitative results, with an average win rate of 91.2% across all test scenarios, speak to the superiority of AlphaCrash over the high-level enemy. The consistent ‘win by crash’ result showcases the agent’s ability to consistently lead adver-

saries into fatal errors, culminating in high-probability crash outcomes.

4.2.2. Learning curves

Reward curves provide critical insights into algorithmic learning stability and effectiveness, enabling performance comparison under uncertainty. Fig. 8 presents the training reward dynamics through mean values and 95% confidence intervals for all three methods.

AlphaCrash demonstrated superior performance with the highest mean reward, characterized by steady growth and narrow confidence intervals that indicate stable, consistent learning. DQN* showed rapid initial reward gains but plateaued, displaying oscillating performance with sharp declines followed by gradual recoveries, ultimately limiting its final effectiveness. Throughout the training, AlphaCrash maintained continuous improvement, ultimately outperforming DQN* in both adaptability and reward optimization. PPO* exhibited early training instability with wide confidence intervals, and despite eventual convergence, achieved lower mean rewards than both DQN* and AlphaCrash.

The superior performance of AlphaCrash, based on the QR-DQN algorithm, can be attributed to the fundamental differences between QR-DQN and the traditional DQN algorithm. DQN relies on optimizing the expected reward for policy updates, while QR-DQN employs quantile regression to estimate the entire reward distribution, providing a richer representation of the underlying dynamics. The learning curves reveal AlphaCrash’s stable performance through progressively narrowing confidence intervals and sustained improvement. This robustness to environmental uncertainty is a direct result of QR-DQN’s ability to model reward distributions more comprehensively. Additionally, QR-DQN offers enhanced stability compared to DQN, which is prone to oscillations and sharp performance drops during training due to its reliance on single-value approximations of the Q-function. The stability of AlphaCrash, as reflected in its smooth and steadily increasing reward curve, suggests that it successfully mitigates these issues by leveraging the distributed quantile estimates to achieve more reliable updates. Another significant advantage of QR-DQN is its effectiveness in handling exploration versus exploitation trade-offs. QR-DQN’s comprehensive reward dis-

Table 3 Performance evaluation results in different test scenarios.

Scenario	AlphaCrash				DQN*				PPO*				Visualization
	Win	Lose	Draw	Win rate(%)	Win	Lose	Draw	Win rate(%)	Win	Lose	Draw	Win rate(%)	
1	97	0	3	97	74	7	19	74	11	49	40	11	
	90	0	10	90	63	9	28	63	6	20	74	6	
2	89	0	11	89	64	11	25	64	4	28	68	4	
	93	0	7	93	66	15	19	66	5	23	72	5	
3	89	0	11	89	69	9	22	69	3	35	62	3	
	96	0	4	96	67	15	18	67	9	30	61	9	
4	96	0	4	96	75	19	6	75	5	23	72	5	
	93	0	7	93	70	17	13	70	7	31	62	7	
5	74	0	26	74	53	30	17	53	3	31	66	3	
6	95	0	5	95	62	29	9	62	6	25	69	6	
Total	912	0	88	91.2	663	161	176	66.3	59	295	646	5.9	

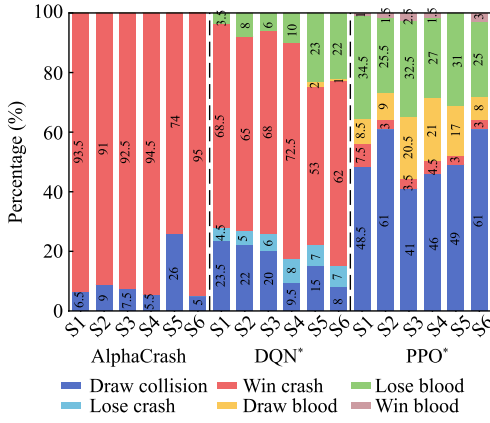


Fig. 7 Win-loss percentages in different test scenarios.

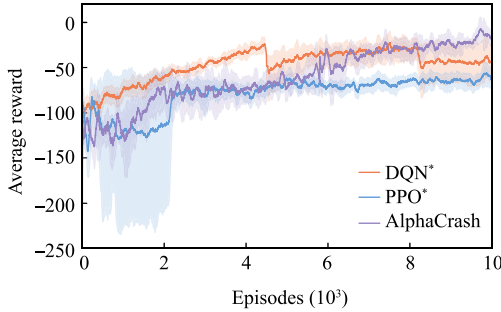


Fig. 8 Training reward curves with mean values and 95% confidence intervals for different methods.

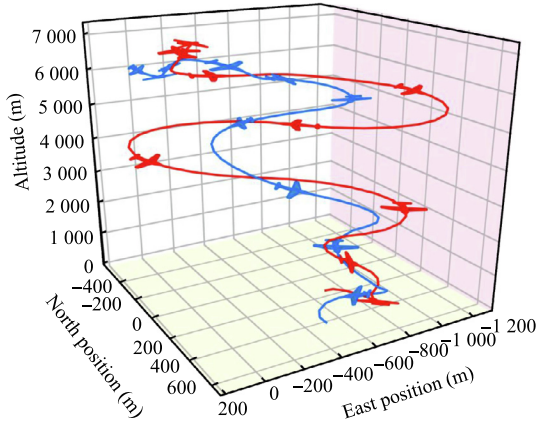


Fig. 9 Complete air combat process of Case 1.

tribution analysis enables more effective exploration and policy optimization. Consequently, AlphaCrash achieves steady performance gains, whereas DQN* shows initial rapid but unstable growth before plateauing due to limited exploration and premature convergence. PPO* lagged behind both AlphaCrash and DQN* in achieving competitive average rewards. Its wide confidence intervals during early training indicate a higher sensitivity to exploration noise or hyperparameter tun-

ing issues. Although PPO* eventually converged, the slower convergence rate and lower mean rewards highlight inefficiencies in adapting to the learning task.

4.3. Strategy analysis

In this section, we comprehensively visualize the experimental sparring trajectories to assess the sparring effects in greater detail. We explore and understand the process of intelligent body sparring and strategic decision-making in depth.

4.3.1. Case study

Two test rounds are randomly selected for demonstration and analysis. Their complete air combat processes are shown in Figs. 9 and 10, and their respective air combat sequential breakdown processes are illustrated in Figs. 11 and 12 as well. In the figures, the blue aircraft represents the high-level enemy strategy detailed in Section 3.8, while the red aircraft represents our trained crash induction agent.

Analysis reveals that the agent has developed a sophisticated crash induction strategy comprising three stages: inducing to close range, spiraling descent, and sudden leveling for the clash.

In the initial inducing to close range stage as shown in Figs. 11(a) and 12(a), the red aircraft actively approaches the blue aircraft, provoking pursuit and engagement. This maneuver induces the blue aircraft to maintain close pursuit, gradually reducing the inter-aircraft distance. Concurrently, the red aircraft descends, guiding the blue aircraft into a lower altitude zone. The spiraling descent stage as shown in Figs. 11(b) and 12(b), initiates a dogfight between the red and blue aircraft. The red aircraft maintains a lead position, forcing the blue aircraft into a tail chase while evading maneuvers to damage the weapon. The red aircraft continues its descent throughout this stage while engaging in the dogfight. The blue aircraft's close pursuit leads it into the predetermined warning altitude ($h_{\text{warning}} = 5$ km). The final stage is sudden leveling for the clash, as shown in Figs. 11(c) and 12(c). The red aircraft maintains its lead position while continuing to descend, with the blue aircraft in full-speed pursuit. As the red aircraft approaches the boundary, it executes a sharp turn and levels out approximately 100 m from the limit. Unable to brake in time, the blue aircraft nosedives past the minimum altitude and crashes. In most games, Blue is defeated at this stage due to a control overshoot caused by untimely braking. Occasionally, Blue brakes early and misses the boundary, as shown in Fig. 12(c). At this point, the red aircraft continues circling close to the minimum altitude while rapidly approaching the Blue aircraft. This maneuver induces the blue aircraft to approach the minimum altitude again during the close chase, eventually resulting in a crash when the brakes are not applied in time.

Through the visualization of the battle process between the red and blue sides, it is observed that the red side adopts a clever strategy of inducing crashes when in fierce pursuit. Initially, the red side engages in close entanglement, then slowly lures the opponent, gradually approaching the boundary. At a critical moment, the red side suddenly and sharply turns to strike. For the remaining undefeated opponents, the red side continually uses boundary-induced maneuvers. This entire

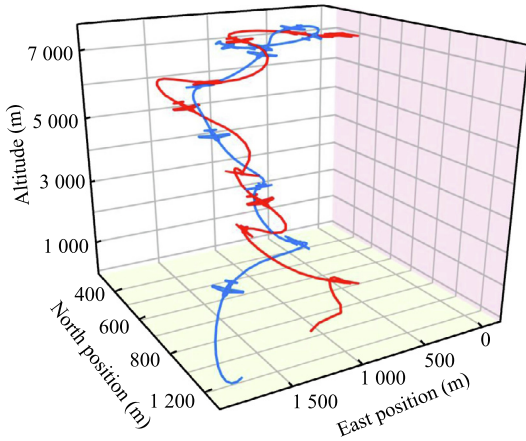


Fig. 10 Complete air combat process of Case 2.

process is vividly and cleverly presented in the visualization, demonstrating the effectiveness of the algorithmic framework formed by the combat algorithm and reward function and validating the agent's advanced air combat capabilities.

4.3.2. Altitude analysis

Furthermore, the altitude changes of the opposing aircraft provide additional insights into the strategy mechanisms. Thus, the strategies are analyzed more intuitively by plotting the altitude change curves.

The altitude data for both aircraft across all successful combats of AlphaCrash against the high-level enemy is depicted in Fig. 13(a), comparing the final and fifth-to-last frames of each round. The vertical axis denotes flight altitude, while the horizontal axis represents the round number. In the fifth-to-last frame, the red aircraft consistently maintains a lower altitude than the blue. However, in the final frame, this relationship inverts: the red aircraft's altitude exceeds that of the blue, with the latter falling below the minimum altitude, indicating a crash. This pattern suggests the implementation of a leveling strategy by AlphaCrash near the boundary, inducing the opponent to dive and crash before effective braking could occur.

The complete altitude profile for a randomly selected air combat is illustrated in Fig. 13(b). Initially, both aircraft exhibit altitude fluctuations. Subsequently, AlphaCrash initiates a sharp descent, with the opponent maintaining a slightly higher altitude in pursuit. During the critical engagement phase, the aircraft alternately occupy the superior altitude position. In the final phase, a distinct altitude separation occurs. AlphaCrash executes a leveling maneuver, while the opponent continues to descend, ultimately below the minimum altitude, signifying a crash.

The complete altitude change chart illustrates the gradual pull-up, the alternating chase-down, and the final maneuvering, where the red side flies flat and the blue side falls fatally. This entire sequence demonstrates once again that the AlphaCrash strategy is a clever and effective method for luring the enemy down in the WVR battle.

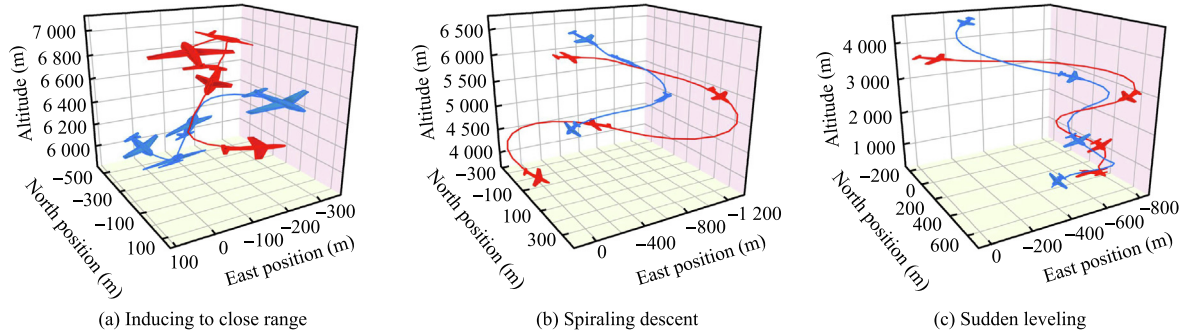


Fig. 11 Sequential breakdown of air combat process in Case 1.

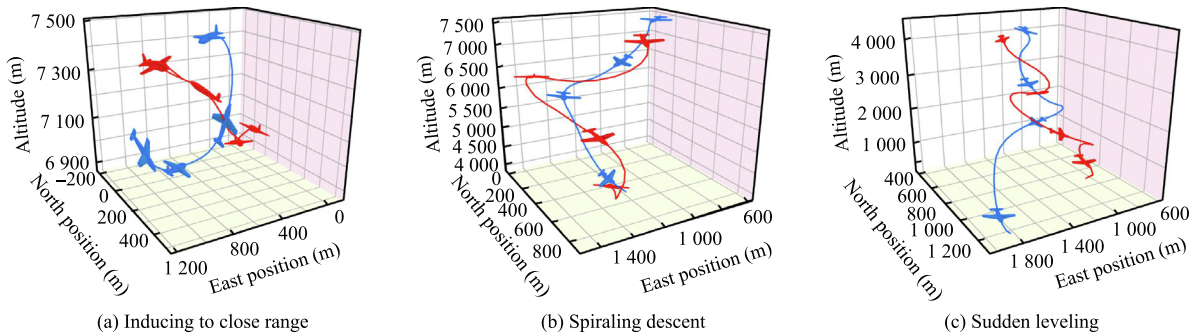


Fig. 12 Sequential breakdown of air combat process in Case 2.

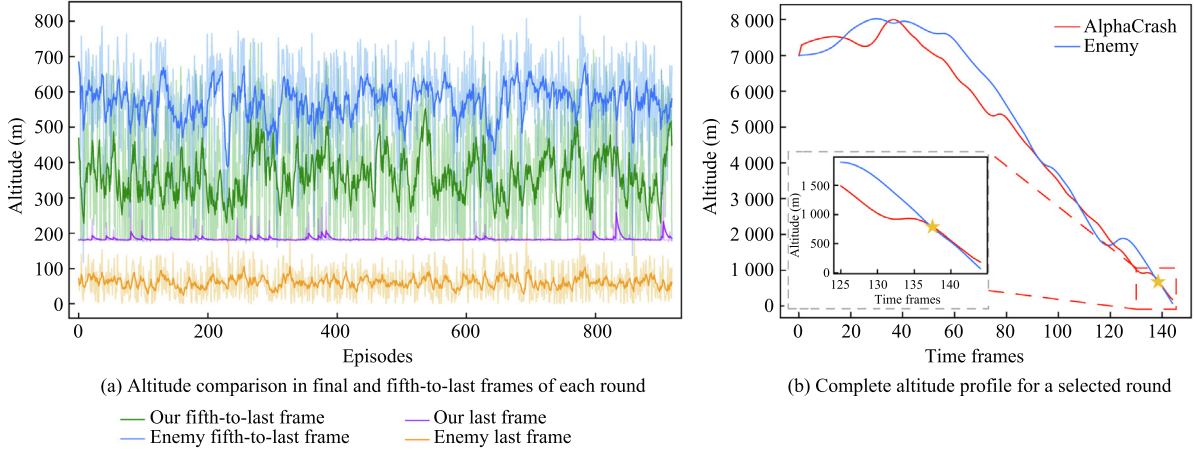


Fig. 13 Altitude variations of red and blue flights during air combat.

4.4. Discussion

The experimental results demonstrate the effectiveness and robustness of the AlphaCrash framework across various WVR air combat scenarios. Achieving an impressive average win rate of 91.2%, AlphaCrash consistently outperformed other traditional RL methods. This performance highlights the efficacy of the distributional RL approach in complex adversarial environments, where decision-making under uncertainty is critical.

A detailed analysis of AlphaCrash’s behavior reveals its unique three-stage crash induction strategy. Initially, the agent engages the adversary in close-range combat, exploiting its pursuit behavior to maintain proximity. In the second phase, a spiraling descent leads the opponent to lower altitudes while preserving the agent’s own positional advantage. Finally, a sudden leveling maneuver near critical altitude boundaries induces the adversary to overshoot, resulting in crashes. This sophisticated tactic highlights the agent’s ability to anticipate and exploit the adversary’s vulnerabilities, demonstrating the practical applicability of its learned strategies.

Despite these achievements, several limitations of the current framework should be acknowledged. First, the simulation environment simplifies real-world conditions by neglecting factors such as sensor noise, environmental variability, and hardware constraints, which are crucial in operational settings. The predefined altitude boundaries and maneuver libraries, while effective for WVR combat, may not generalize well to other scenarios, such as multi-agent engagements or BVR operations. Additionally, the reward function relies heavily on pre-specified parameters, such as ideal distances and warning altitudes, which could limit the agent’s adaptability to novel adversarial tactics or combat dynamics.

Another limitation lies in the inherent characteristics of the QR-DQN algorithm, which serves as the backbone of the AlphaCrash framework. While distributional RL enhances decision-making by modeling the entire return

distribution, it introduces increased computational complexity compared to traditional Q-learning methods. This added complexity may hinder scalability, especially in scenarios involving larger state-action spaces or multi-agent environments. Furthermore, QR-DQN’s reliance on discrete quantiles to represent return distributions can lead to approximation errors if the number of quantiles is insufficient or if the reward distribution exhibits extreme variations. Although the algorithm effectively handles uncertainty and adversarial strategies in the WVR domain, its robustness in highly stochastic or non-stationary environments remains to be validated.

To address these limitations, future work should focus on enhancing the realism and flexibility of the simulation environment. Incorporating noise, environmental effects, and adversarial uncertainties would provide a more accurate representation of real-world challenges. Furthermore, developing adaptive reward functions capable of dynamically adjusting priorities based on evolving combat conditions would enhance the agent’s tactical versatility. Moreover, introducing additional maneuver objectives, such as weapon optimization and evasive tactics, could diversify the agent’s strategic repertoire. Finally, addressing the inherent limitations of QR-DQN is essential for ensuring the scalability and robustness of the framework. Future research could explore adaptive quantile selection mechanisms or continuous return distribution modeling to reduce approximation errors in highly stochastic or non-stationary environments.

5. Conclusions

This study investigates crash induction strategies for WVR air combat scenarios. Key challenges in this field include bridging the research-practice gap and incorporating expert pilot knowledge. We present AlphaCrash, a distributional RL agent designed for WVR air combat engagement. The proposed framework implements an enhanced QR-DQN methodology, improving agent performance under tactical uncertainty. The combat agent development integrates

expert pilot knowledge through comprehensive reward functions—incorporating spatial, kinematic, and boundary parameters, and utilizing high-level expert strategies to enrich the learning adaptability. The framework also contains comprehensive flight state and discrete representations for maneuver design, enabling practical crash induction applications.

Through extensive and realistic comprehensive simulations and strategic analysis, our method demonstrates effectiveness and robustness across a wide range of combat scenarios. The results of the quantitative study show an average AlphaCrash win rate of 91.2% across all scenarios tested, highlighting the superiority of our methodology over the current high level. Recurring "Crash to Win" results demonstrate the agent's ability to consistently manipulate opponents into making fatal mistakes, leading to frequent crash situations. Visualization of the combat process shows that AlphaCrash learned a clever three-phase sparring strategy during training: induced close-range attacks, spiral descents, and decisive sudden-level flights. The strategy is further validated by flight altitude analysis. It reveals that the agent will repeatedly approach the enemy aircraft quickly before circling to lure it, ultimately causing it to break out of overshoot and crash. In summary, AlphaCrash targets real-world application scenarios and demonstrates a significant advancement in autonomous air combat strategy. It utilizes distributional RL to effectively induce enemy crash failure. Future work will aim to enhance the strategy's robustness in more diverse and complex real-world scenarios and further improve the agent's multi-scenario decision-making capabilities.

Credit author statement

Xuefeng Liang: Writing - original draft, Investigation, Software, Methodology, Writing - review & editing. **Zetian-Hu:** Writing - original draft, Methodology, Validation, Conceptualization. **Xiaochuan-You:** Methodology, Writing- review & editing. **Jun-zhang:** Data curation, Resources. **Chengcheng-Ma:** Resources, Data curation.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

The work was supported by the National Key R&D Program of China (No. 2021YFB3300602).

Appendix A. Hyperparameter configurations

This section provides detailed hyperparameter settings for AlphaCrash, DQN*, and PPO* used in our experiments. The parameters are summarized in Table A1.

Table A1. Hyperparameter settings for RL algorithms.

Hyperparameter	AlphaCrash (QR-DQN)	DQN*	PPO*
Network architecture	3 layers, 64 neurons/layer	3 layers, 64 neurons/layer	Policy & Value: 2 layers, 256 neurons/layer
Replay buffer capacity	600 000	600 000	
Prioritized Experience Replay (PER)	$\alpha = 0.6, \beta = 0.4$	$\alpha = 0.6, \beta = 0.4$	
Batch size	32	64	128
Learning rate	0.000 1	0.000 1	0.000 3
Optimizer	Adam	Adam	Adam
Discount factor γ	0.9	0.9	0.99
Target network update	Soft update ($\tau = 0.01$)	Soft update ($\tau = 0.01$)	
Exploration strategy	ϵ -greedy (0.9 to 0.1, decay 15 000 steps)	ϵ -greedy (0.9 to 0.1, decay 5 000 steps)	
Clipping parameter			0.2
Entropy coefficient			0.01
Value Loss coefficient			0.5
Generalized advantage estimation λ			0.95
Quantile Regression (QR-DQN)	20 quantiles, $\kappa = 1.0$		

Appendix B. Simulation parameters

The parameters used for the WVR air combat simulation scenarios are outlined in Table B1.

Table B1. Simulation Parameters.

Parameter	Value
State update time step (ms)	1
Decision-making time step (ms)	100
Maximum duration per episode (s)	120
Maximum flight speed (m/s)	800
Minimum flight speed (m/s)	100
Maximum Overload (G)	9
Minimum Overload (G)	−3
Maximum sideslip angular velocity ($^{\circ}$ /s)	30
Maximum pitch angular velocity ($^{\circ}$ /s)	60

References

1. Jordan J. The future of unmanned combat aerial vehicles: an analysis using the three horizons framework. *Futures* 2021;134:102848.
2. Zhang JD, Wang DH, Yang QM, et al. Loyal wingman task execution for future aerial combat: A hierarchical prior-based reinforcement learning approach. *Chin J Aeronaut* 2024;37(5):462–81.

3. Kumar S. A brief review on unmanned combat aerial vehicle (U.C.A.V). *SSRN Journal* 2020. <https://doi.org/10.2139/ssrn.3593220>.
4. Hu JW, Wang LH, Hu TM, et al. Autonomous maneuver decision making of dual-UAV cooperative air combat based on deep reinforcement learning. *Electronics* 2022;**11**(3):467.
5. Wang XW, Wang YH, Su XC, et al. Deep reinforcement learning-based air combat maneuver decision-making: Literature review, implementation tutorial and future direction. *Artif Intell Rev* 2023;**57**(1):1.
6. Wang LX, Zheng SZ, Piao HY, et al. Tube-based robust reinforcement learning for autonomous maneuver decision for UCAVs. *Chin J Aeronaut* 2024;**37**(7):391–405.
7. Park H, Lee BY, Tahk MJ, et al. Differential game based air combat maneuver generation using scoring function matrix. *Int J Aeronaut Space Sci* 2016;**17**(2):204–13.
8. Poropudas J, Virtanen K. Game-theoretic validation and analysis of air combat simulation models. *IEEE Trans Syst Man Cybern Part A Syst Hum* 2010;**40**(5):1057–70.
9. Rardin RL, Uzsoy R. Experimental evaluation of heuristic optimization algorithms: A tutorial. *J Heuristics* 2001;**7**(3):261–304.
10. Chai RQ, Liu DR, Liu TH, et al. Deep learning-based trajectory planning and control for autonomous ground vehicle parking maneuver. *IEEE Trans Autom Sci Eng* 2023;**20**(3):1633–47.
11. McGrew JS, How JP, Williams B, et al. Air-combat strategy using approximate dynamic programming. *J Guid Contr Dyn* 2010;**33**(5):1641–54.
12. Chai RQ, Tsourdos A, Savvaris A, et al. Real-time reentry trajectory planning of hypersonic vehicles: A two-step strategy incorporating fuzzy multiobjective transcription and deep neural network. *IEEE Trans Ind Electron* 2020;**67**(8):6904–15.
13. Chai RQ, Tsourdos A, Savvaris A, et al. Six-DOF spacecraft optimal trajectory planning and real-time attitude control: A deep neural network-based approach. *IEEE Trans Neural Netw Learn Syst* 2020;**31**(11):5005–13.
14. Ernest N, Carroll D. Genetic fuzzy based artificial intelligence for unmanned combat aerial vehicle control in simulated air combat missions. *J Def Manag* 2016;**6**(1): 2167-0374.
15. Geng WX, Kong FE, Ma DQ. Study on tactical decision of UAV medium-range air combat. In: *The 26th chinese control and decision conference (2014 CCDC)*. Piscataway: IEEE Press; 2014.
16. Huang CQ, Wei ZL, Yang YZ, et al. Knowledge acquisition for the air combat based on GWO. *J Phys Conf Ser* 2019;**1325**(1):012078.
17. Silver D, Hubert T, Schrittwieser J, et al. A general reinforcement learning algorithm that Masters chess, shogi, and Go through self-play. *Science* 2018;**362**(6419):1140–4.
18. Kaufmann E, Bauersfeld L, Loquercio A, et al. Champion-level drone racing using deep reinforcement learning. *Nature* 2023;**620**(7976):982–7.
19. Ma XT, Xia L, Zhao QC. Air-combat strategy using deep Q-learning. *2018 Chinese automation congress (CAC)*. Piscataway: IEEE Press; 2018. p. 3952–7.
20. Xu J, Guo Q, Xiao L, et al. Autonomous decision-making method for combat mission of UAV based on deep reinforcement learning. *2019 IEEE 4th advanced information technology, electronic and automation control conference (IAEAC)*. Piscataway: IEEE Press; 2019. p. 538–44.
21. Yang QM, Zhang JD, Shi GQ, et al. Maneuver decision of UAV in short-range air combat based on deep reinforcement learning. *IEEE Access* 2019;**8**:363–78.
22. Zhang HP, Wei YJ, Zhou H, et al. Maneuver decision-making for autonomous air combat based on FRE-PPO. *Appl Sci* 2022;**12**(20):10230.
23. Pope AP, Ide JS, Mićović D, et al. Hierarchical reinforcement learning for air combat at DARPA's AlphaDogfight trials. *IEEE Trans Artif Intell* 2023;**4**(6):1371–85.
24. Chai JJ, Chen WZ, Zhu YH, et al. A hierarchical deep reinforcement learning framework for 6-DOF UCAV air-to-air combat. *IEEE Trans Syst Man Cybern Syst* 2023;**53**(9):5417–29.
25. Bae JH, Jung H, Kim S, et al. Deep reinforcement learning-based air-to-air combat maneuver generation in a realistic environment. *IEEE Access* 2023;**11**:26427–40.
26. Owen M, Beard RW, McLain TW. Implementing Dubins airplane paths on fixed-wing UAVs. Dordrecht: Springer, Netherlands; 2015.p.1677-701.
27. Jaderberg M, Mnih V, Czarnecki WM, et al. Reinforcement learning with unsupervised auxiliary tasks. arXiv preprints: 1611.05397; 2016.
28. Mnih V, Kavukcuoglu K, Silver D, et al. Human-level control through deep reinforcement learning. *Nature* 2015;**518**(7540):529–33.
29. Dabney W, Rowland M, Bellemare M, et al. Distributional reinforcement learning with quantile regression. *Proceedings of the AAAI conference on artificial intelligence*. Reston: AIAA; 2018.
30. Sun LK, Qiu HX, Wang YZ, et al. Autonomous UAV maneuvering decisions by refining opponent strategies. *IEEE Trans Aerosp Electron Syst* 2024;**60**(3):3454–67.
31. Sun TY, Tsai SJ, Lee YN, et al. The study on intelligent advanced fighter air combat decision support system. *2006 IEEE international conference on information reuse & integration*. Piscataway: IEEE Press; 2006. p. 39–44.
32. Mnih V, Kavukcuoglu K, Silver D, et al. Playing atari with deep reinforcement learning. arXiv preprint: 1312.5602; 2013.
33. Schulman J, Wolski F, Dhariwal P, et al. Proximal policy optimization algorithms. arXiv preprint: 1707.06347; 2017.